From Code to Theorem

The `mistyR` $\longleftrightarrow$ *Music of the Primes* Bridge

Ali Taqi*

July 12, 2026

Abstract

`mistyR` (2022) is an R implementation of Musical Set Theory: a core library for spelling notes, intervals, and scales, plus two score-import modules (`kern`, MIDI) feeding a stylometry suite. *Music of the Primes* (MOTP, 2026) is a book manuscript whose Chapter 2, *Chromatic-Accidental Arithmetic*, develops the same theory formally: a musical alphabet $\mathcal{L} \cong \mathbb{Z}_7$, an accidental norm $|\alpha|$, a family of white-key distance rules d_2, d_3, d_5, d_n, and a transposition-invariance theorem. This paper explains, construct by construct, how the two artifacts are the same mathematics written twice — once as executable R, once as definitions and theorems — and documents the places where each side outruns the other. The direction of the arrow matters: the code came first, and the book is its *retroactive formalization*. We give the evidence (shared vocabulary fossils, identical algorithmic structure, a theorem that is precisely a function's correctness argument), derive the central identity `.WKsemitones` $\equiv d_n$, and map the full pipeline from Chapter 2's arithmetic up through the modality stylometry that runs on real scores.

Contents

*Prepared 2026-07-12 as part of the bridge-artifact series (with Claude). Companion artifacts: `docs/motp-concordance.md` (the maintained mapping) and `docs/motp-bridge-todo.md` (the ledger). This PDF is the expository snapshot of the 2026-07-10 cross-reference audit.

1 Introduction: Two Artifacts, One Theory

Between 2022 and 2026, the same body of musical mathematics was written down twice.

The first time, it was written as software. `mistyR` is an R package that spells scales in any key, computes scale degrees of arbitrary notes against arbitrary tonics, resolves enharmonics, and parses `.krn` and `.mid` scores into a shared tidy "piece frame" on which a suite of stylometric analyses runs. Its core library (`R/`) is small — six files — and every function in it encodes a fact about how Western pitch notation works.

The second time, it was written as mathematics. Chapter 2 of *Music of the Primes*, *Chromatic-Accidental Arithmetic*, builds the same machinery from first principles: the musical alphabet as $\mathbb{Z}_7$, accidentals as a signed norm, step distances decided by the black-key layout of the piano, and a preservation theorem that reduces all chromatic spelling problems to white-key problems.

Neither artifact cites the other, but they did not arise independently: the book is the code's *retroactive formalization*. The evidence is threefold, and this paper is organized around it:

1. **Identical algorithmic skeletons** (§6–§8). The book's n-degree distance rule and the code's `.WKsemitones()` are the same computation; the book's Accidental-Transpositional Preservation Principle is, line for line, the correctness argument of the code's `scale()` function.
2. **Vocabulary fossils** (§12). The code's original, idiosyncratic word for accidentals — *incidental* — survives inside two of the book's formal definitions. The code's ASCII workaround for double-sharps (`x`) fossilized into the book's typography.
3. **Matched asymmetries** (§11, §15). Where the book has theory the code lacks (quality dyads/triads), there is *no* implementation anywhere in the repository — because that section is new mathematics, not formalization. Where the code has machinery the book lacks (double accidentals, enharmonic simplification, score parsing), the book's scope simply has not caught up.

How to read this paper. Each section takes one theoretical construct, quotes the book's formulation, exhibits the code that implements it, and explains the correspondence — including, where they exist, the small divergences of convention that make the mapping non-obvious at first sight. Book results are quoted in *Book Definition / Distance Rule / Theorem* environments, keyed by their titles in `chapters/chapter2.tex` (never by page number, so the mapping survives edits). Original observations are set as *Bridge Observations*. Appendix A collects the entire translation key in one table.

The two repositories.

	Book	**Code**
Location	`music-of-the-primes/`	`mistyR/`
Scope here	Ch. 2 *Chromatic-Accidental Arithmetic*	`R/` core; modules where noted
Objects	$\mathcal{L}$, Λ_k, $\lvert\alpha\rvert$, d_n, $\overline{\mathbf{Blk}}$, Δ/∇, $\mathcal{F}$	`data.R`, `accidental.R`, `interval.R`, `scale.R`, `harmony.R`, `analysis.R`
Born	2026 (manuscript, active)	2022 (prototype, revived 2026)

2 The Musical Alphabet: $\mathcal{L} \cong \mathbb{Z}_7$

Book Definition 2.1 (Musical Alphabet)**.** The musical alphabet is a set of letters $\mathcal{L} = \{A, B, \ldots, F, G\}$, where the letter after G loops back to A. The book immediately notes $\mathcal{L} \cong \mathbb{Z}_7$: fixing $A = 0, B = 1, \ldots, G = 6$, looping is addition in the group of integers modulo 7.

The code declares the same object as a global in `data.R`:

```
MUS_ALPH <- LETTERS[1:7]          # A B C D E F G
mod7 <- function(x) { x %% 7 }    # arithmetic modulo 7
note_index <- function(note){ which(MUS_ALPH == note) }
```

The isomorphism is realized by `note_index()` — except for one convention divergence that echoes through the entire codebase:

Bridge Observation 2.1 (The off-by-one dance). The book works in $\mathbb{Z}_7 = \{0, \ldots, 6\}$ with $A = 0$. R is 1-indexed, so the code stores $A = 1, \ldots, G = 7$ and must conjugate every modular operation through the shift $x \mapsto x - 1$:

```
.ALPH_RANGE <- function(i, length){    # wrapped range of letters
  range <- i:(i + (length - 1))
  range <- mod7(range - 1)             # shift down, reduce mod 7,
  range + 1                            # shift back up
}
.IDX_STATIC <- function(range){ mod7(range - 1) + 1 }
```

The recurring pattern `mod7`$(x-1)+1$ is the 1-indexed shadow of the book's clean $\mathbb{Z}_7$ arithmetic. Reading the book as "the version of the code where the indexing finally got cleaned up" is historically accurate: the formalization chose 0-indexing precisely because the prototype's 1-indexed gymnastics were noise.

3 Letter Degrees and Sequences

Book Definition 3.1 (k-Degree Letter; k-Degree Letter Sequence)**.** For $L \in \mathcal{L}$, the k^{th} degree letter is $\Lambda_k(L) = L \oplus (k-1)$, the terminus of the sequence

$$\mathbb{L}_k[L] = \{L,\ L \oplus 1,\ \ldots,\ L \oplus (k-1)\},$$

where $\oplus$ is letter addition *with looping*. The sequence has k letters including the start, matching the musician's 1-indexed convention: the "fifth of F" is $\Lambda_5(F) = C$, reached after four steps.

The code implements the operator and the sequence separately, in `scale.R` and `data.R`:

```
`%STEPUP%` <- function(scale, note){       # one application of L (+) 1
  idx <- note_index(note)
  scale[mod7(7 + idx) + 1]
}
`%STEP+%` <- function(note, steps){        # Lambda_k: iterate k-1 times
  for(i in 1:steps){ note <- MUS_ALPH %STEPUP% note }
  note
}
# The sequence L_k[L] is a wrapped index range:
# MUS_ALPH[.ALPH_RANGE(note_index(L), k)]
```

So $\Lambda_k(L) =$ `L %STEP+% (k - 1)` and $\mathbb{L}_k[L] =$ `MUS_ALPH[.ALPH_RANGE(note_index(L), k)]`. Both sides share the same 1-indexed *musical*-degree convention (k letters, $k-1$ steps) even though the book's underlying letter arithmetic is 0-indexed — the $(k-1)$ in the book's definition is exactly the `degree_n - 1` that appears wherever the code builds intervals:

```
# from .intervalNote() in interval.R:
white <- tonic %STEP+% (degree_n - 1)    # Lambda_k(tonic) = tonic (+) (k-1)
```

Bridge Observation 3.1 (Rule of 9 is book-only). The book proves $\Lambda_k(L) = L' \iff \Lambda_{9-k}(L') = L$ (fifths pair with fourths, sixths with thirds, sevenths with seconds). The code never inverts an interval — it always computes degrees directly — so this theorem has no code counterpart. It is one of the first places the formalization *adds* structure rather than transcribing it (see §15).

The heptatonic scale template $\mathcal{S}_7[L] = \mathbb{L}_7[L]$ ("the musical alphabet, beginning at L") is the code's `.baseScale()` (`harmony.R`):

```
.baseScale <- function(tonic){
  MUS_ALPH[.ALPH_RANGE(note_index(tonic), length = 7)]
}
```

4 Pitch Labels and Accidental Arithmetic

Book Definition 4.1 (Note-Letter; Accidentals)**.** An accidental is $\alpha \in \mathcal{A} = \{\flat, \natural, \sharp\}$. A pitch label (note-letter) pairs a letter with an accidental: $\pi = (L, \alpha) = L^\alpha \in \mathcal{L}^* = \mathcal{L} \times \mathcal{A}$. Flats and sharps neutralize: $(L^\flat)^\sharp = L^\natural = (L^\sharp)^\flat$.

In the code, a pitch label is simply a string (`"F#"`, `"B-"`), and the pair structure is recovered by two projections in `accidental.R`:

```
.dropAccidental   <- function(note){ substring(note, 1, 1) }  # Letter(pi)
.detectAccidental <- function(note){ ... }                    # Acc(pi), via ACC_
    HASH
```

These are exactly the book's getters $\mathrm{Letter}(\pi)$ and $\mathrm{Acc}(\pi)$ (macros `\Let`, `\Acc` in `preamble/commands.tex`). Applying an accidental — the book's $\pi \odot \alpha$ — is the infix `%ACC%` wrapping `.addAccidental()`, and the neutralization axiom is literally a branch of that function:

```
.addAccidental <- function(note, accidental){
  ...
  if(accidental == "flat"){
    if(note_acc == "flat")         { accidental <- "&" }  # bb (double flat)
    else if(note_acc == "natural") { accidental <- "b" }
    else if(note_acc == "sharp")   { accidental <- ""  }  # neutralization!
  }
  ...
}
```

Book Distance Rule 4.2 (Semitonal-Accidental Norm)**.** $d_\alpha : \mathcal{L} \times \mathcal{L}^* \to \{-1, 0, 1\}$ measures the distance between a fixed letter and an accidentalized copy of itself: $d_\alpha(L, L^\alpha) = |\alpha|$ where $|\flat| = -1$, $|\natural| = 0$, $|\sharp| = +1$.

The code's version is `.accidentalSemitones()` — and here the code *extends* the book:

```
.accidentalSemitones <- function(x, inverse = T){
  ACCIDENTAL <- c("&", "-", "", "#", "x")
  VALUE      <- c(-2,  -1,  0,  1,   2)
  if(inverse) { return(ACCIDENTAL[which(VALUE == x)]) }
  VALUE[which(ACCIDENTAL == x)]
}
```

Bridge Observation 4.1 (The norm, extended to doubles). The code treats double accidentals as first-class citizens: the norm runs over $\{-2, \ldots, +2\}$ with symbols `&` (double flat) and `x` (double sharp), and the function is invertible in both directions — it is used *norm-inverse-first* throughout the spelling engine (given a semitone discrepancy, produce the accidental). The book's $\mathcal{A} = \{\flat, \natural, \sharp\}$ truncates to singles even though $\flat\flat$ and $\sharp\sharp$ appear in its own accidental-family tables — extending the Semitonal-Accidental Norm to ± 2 is a pending book-side item (ledger B14). This is the cleanest example of the code being *ahead* of its own formalization.

Note also the flat symbol: the code's canonical flat is the kern convention `-` (so $B\flat$ is `"B-"`), with letter `b` accepted on input. The book writes $\flat$ only. This single divergence explains most of the surface-level unfamiliarity when reading one artifact with the other's eyes.

5 Color: Black, White, and Den

Book Definition 5.1 (Black Flat-/Sharp-Supported Letters)**.** $L \in \overline{\mathbf{Blk}}[\flat] \iff \mathrm{Color}(L^{\flat}) = \mathbf{Blk}$, giving $\overline{\mathbf{Blk}}[\flat] = \{A, B, D, E, G\}$ with complement $\overline{\mathbf{Wht}}[\flat] = \{C, F\}$. Dually, $\overline{\mathbf{Blk}}[\sharp] = \{A, C, D, F, G\}$ with complement $\overline{\mathbf{Wht}}[\sharp] = \{B, E\}$.

This is a two-column boolean table in `data.R` — arguably the single most load-bearing data structure in the package:

```
ALPH_HASH <- function(){
  MUS_ALPH <- LETTERS[1:7]                #  A B C D E F G
  HAS_FL <- as.logical(c(1,1,0,1,1,0,1))  #  Blk[flat]:  A B . D E . G
  HAS_SH <- as.logical(c(1,0,1,1,0,1,1))  #  Blk[sharp]: A . C D . F G
  data.frame(LETTER = MUS_ALPH, HAS_FL, HAS_SH)
}
```

The predicate Color is implemented directly:

```
.isBlackNote <- function(note){
  note_acc    <- .detectAccidental(note)
  note_letter <- .dropAccidental(note)
  if(note_acc == "flat") { return(note_letter %in% HAS_FLATS)  }
  else if(note_acc == "sharp"){ return(note_letter %in% HAS_SHARPS) }
  FALSE
}
```

Bridge Observation 5.1 (Den keys are a book-native abstraction). The book goes one step further and intersects the two supports:

$$\overline{\mathbf{Den}} = \overline{\mathbf{Blk}}[\flat] \cap \overline{\mathbf{Blk}}[\sharp] = \{A, D, G\} \quad \text{(mnemonic: "A Den 'G'")},$$

with complement the *white neighbours* $\overline{\mathbf{WhtNeb}} = \{B, C, E, F\}$. The code stores `HAS_FL` and `HAS_SH` but *never intersects them* — no function computes or uses $\overline{\mathbf{Den}}$. The den/neighbour

classification is the formalization discovering structure the prototype had in its data but never named. (Adding `DEN_KEYS` and `WHITE_NEIGHBOURS` globals is ledger A8 — cheap, and it would let code comments cite the book.)

6 Distance Rules and the White-Key Engine

This section contains the strongest single piece of evidence that book and code are one theory. We first quote the book's two rules, then show they compose into exactly the code's `.WKsemitones()`.

Book Distance Rule 6.1 (Step Distance)**.** $d_2 : \mathcal{L} \to \{1, 2\}$ gives the semitone distance from a letter to its right neighbour $\Lambda_2 = L \oplus 1$:

$$d_2(L) = \begin{cases} 1, & \Lambda_2 \in \overline{\mathbf{Wht}}[\flat] \\ 2, & \Lambda_2 \in \overline{\mathbf{Blk}}[\flat] \end{cases} \qquad \text{equivalently} \qquad d_2(L) = \begin{cases} 1, & L \in \overline{\mathbf{Wht}}[\sharp] \\ 2, & L \in \overline{\mathbf{Blk}}[\sharp]. \end{cases}$$

Book Distance Rule 6.2 (n-Degree Distance)**.** For $L \in \mathcal{L}$ with n^{th}-degree letter $\Lambda_n = L \oplus (n-1)$, the distance accumulates step by step:

$$d_n(L) = d(L, \Lambda_n) = \sum_j d_2(\Lambda_j) = \sum_j d(\Lambda_j, \Lambda_{j+1}),$$

the sum running over the $n-1$ steps from L to Λ_n.[1]

Now the code, from `interval.R`, verbatim:

```
# Find the white-key range number of semitones between
# a note and a given scale degree
.WKsemitones <- function(note, degree){
  note_idx     <- note_index(note)
  range        <- .ALPH_RANGE(note_idx, degree)
  letter_range <- ALPHABET[range,]
  # Get the number of flats in range
  FLATS  <- sum(letter_range[2:degree,]$HAS_FL)
  # Get the number of white keys in the range (exclude tonic)
  WHITES <- degree - 1
  semitones <- FLATS + WHITES
  semitones
}
```

Bridge Observation 6.1 (The central identity: `.WKsemitones` $\equiv d_n$). Write the letters from L to its n^{th} degree as $\Lambda_0 = L, \Lambda_1, \ldots, \Lambda_{n-1} = \Lambda_n(L)$. By the Step Distance rule (flat-side criterion), each step contributes

$$d_2(\Lambda_j) = 1 + \left[\Lambda_{j+1} \in \overline{\mathbf{Blk}}[\flat]\right],$$

i.e. one semitone always, plus one more exactly when the *arrival* letter is flat-supported. Summing the $n-1$ steps:

$$d_n(L) \;=\; \underbrace{(n-1)}_{\text{one per step}} \;+\; \underbrace{\#\{\, j \in \{1, \ldots, n-1\} : \Lambda_j \in \overline{\mathbf{Blk}}[\flat] \,\}}_{\text{flat-supported arrivals}}.$$

[1] The book's display writes the summation bounds as $j = 0$ to $n-1$, which would take n steps; since $\Lambda_n = L \oplus (n-1)$, the intended count is $n-1$ steps ($j = 0$ to $n-2$). The code resolves the wobble unambiguously — it counts `degree - 1` steps — and the book-side cleanup is part of the distance-family naming item, ledger B7.

The code computes precisely these two terms: `WHITES = degree - 1` is the first, and `FLATS = sum(letter_range[2:degree,]$HAS_FL)` — the count of flat-supported letters in the range *excluding the tonic*, i.e. among the arrival letters $\Lambda_1, \ldots, \Lambda_{n-1}$ — is the second. The function is the theorem; `HAS_FL` is $\overline{\mathbf{Blk}}[\flat]$; the exclusion of row 1 is the exclusion of Λ_0 from the arrival set.

Note which criterion the code chose: the *flat-side* case of Rule 6.1 only. The book's dual sharp-side formulation ($d_2(L) = 2 \iff L \in \overline{\mathbf{Blk}}[\sharp]$) appears nowhere in the code — it is added symmetry, discovered at formalization time.

The specialized rules are then corollaries on both sides. The book defines the skip distance $d_3 : \mathcal{L} \to \{3, 4\}$ (two steps; range constrained because no two consecutive letters lie in $\overline{\mathbf{Wht}}[\flat] = \{C, F\}$ — "we cannot find 3 consecutive white keys on a piano") and the triadic distance $d_5 : \mathcal{L} \times \mathcal{L} \to \{6, 7\}$ (two skips; only diminished and perfect fifths occur between white keys). The code never defines d_3 or d_5 separately — it gets them for free as `.WKsemitones(note, 3)` and `.WKsemitones(note, 5)`, exactly as the book gets them as instances of d_n.

Book rule	**Type**	**Range**	**Code realization**
d_2 (Step)	$\mathcal{L} \to \{1, 2\}$	step	`.WKsemitones(L, 2)`
d_3 (Skip)	$\mathcal{L} \to \{3, 4\}$	third	`.WKsemitones(L, 3)`
d_5 (Triadic)	$\mathcal{L}^2 \to \{6, 7\}$	fifth	`.WKsemitones(L, 5)`
d_n (n-Degree)	$\mathcal{L}^2 \to \mathbb{Z}_{12}$	any	`.WKsemitones(L, n)`
d_α (Accidental Norm)	$\mathcal{L} \times \mathcal{L}^* \to \{-1, 0, 1\}$	vertical	`.accidentalSemitones()` (± 2)
$d_\square$ (Octave)	$12k$	octaves	(not needed; octave-free note space)

7 The Chromatic Algorithm: Spelling Any Interval

The book's "Chromatic Algorithm" — described, memorably, as *the music-theory version of borrowing, like in 3rd-grade addition* — spells an arbitrary interval in two moves: lay down the white-key skeleton, then correct with an accidental whose norm makes up the semitone difference. This is, structurally and in order of operations, the code's `.intervalNote()`:

```
.intervalNote <- function(tonic, degree){
  degree_n        <- parse_number(degree)
  semitones       <- SCALE_DEGREES[[degree]]        # target: q_k in Z12
  white           <- tonic %STEP+% (degree_n - 1)   # skeleton: Lambda_k
  semitones_white <- .WKsemitones(tonic, degree_n)  # skeleton's size: d_n
  accidental <- .accidentalSemitones(semitones - semitones_white)
  paste(white, accidental, sep = "")                # correction: |alpha|
}
```

Here `SCALE_DEGREES` (`data.R`) is the book's interval table $q_k \in \mathbb{Z}_{12}$ — the map from degree labels to semitone counts (`"1"` $\mapsto 0$, `"b3"` $\mapsto 3$, `"3"` $\mapsto 4$, `"5"` $\mapsto 7$, ...).

Worked example (both notations). Spell the major third of D.

- *Book:* the target is $q_{\mathrm{M3}} = 4$ semitones. The skeleton is $\Lambda_3(D) = F$, with $d_3(D) = d_2(D) + d_2(E) = 2{+}1 = 3$ (since $E \in \overline{\mathbf{Blk}}[\flat]$ but $F \in \overline{\mathbf{Wht}}[\flat]$). Borrow the difference: $4{-}3 = 1 = |\sharp|$, so the answer is $F^\sharp$.

- *Code:* `SCALE_DEGREES[["3"]]` $= 4$; `D %STEP+% 2` $=$ `"F"`; `.WKsemitones("D", 3)` $= 1+2 =$ 3 (one flat-supported arrival, E, plus two steps); `.accidentalSemitones(4 - 3)` $=$ `"#"`; result `"F#"`.

Same skeleton, same deficit, same correction — and for the minor third (`"b3"`, target 3) the deficit is 0 and both sides answer $F^\natural$.

8 The Preservation Principle is `scale()`'s Correctness Proof

Book Theorem 8.1 (Accidental-Transpositional Preservation Principle)**.** Let $L, L_* \in \mathcal{L}$ be unaccidentalized letters with $\tilde{d}(L, L_*) = k$ semitones, where $\tilde{d}$ is the degree-aware distance (assuming the correct form d_j for $L_* = \Lambda_j(L)$). Fixing $\alpha \in \mathcal{A}$ and accidentalizing *both* letters by α preserves the distance:

$$\tilde{d}(L, L_*) = k \iff \tilde{d}(L^\alpha, L_*^\alpha) = k.$$

The book calls this a *sufficiency result on the white keys*: every spelling problem in $\mathcal{L}^*$ reduces to one in $\mathcal{L}$.

Now read `scale()` (`scale.R`):

```
scale <- function(tonic, scale_degrees){
  note_letter <- .dropAccidental(tonic)
  accidental  <- .detectAccidental(tonic)
  # (1) Solve the problem on the white keys
  base_scale <- .scaleBase(note_letter, scale_degrees)
  if(accidental == "natural"){ return(base_scale) }
  # (2) Transport the solution by the tonic's accidental
  purrr::map_chr(base_scale, .addAccidental, accidental)
}
```

Bridge Observation 8.1 (A theorem that is a function). The function *is* the theorem's proof strategy, executed: strip the tonic's accidental (step into $\mathcal{L}$), spell the entire scale on white keys via the Chromatic Algorithm (`.scaleBase` maps `.intervalNote` over the degree vector), then apply the tonic's accidental to *every* note of the result (transport back to $\mathcal{L}^*$). The output is correct *if and only if* same-accidental transposition preserves all the interval distances — which is exactly what the Preservation Principle asserts. The theorem is the correctness argument `scale()` always needed; the code is the algorithmic content the theorem always had. So `major_scale("E-")` works by spelling the white-key E major scale and flattening all seven notes.

The same white-key-sufficiency reduction runs in the opposite direction in `scaleDegree()` (`harmony.R`), which classifies an arbitrary note against an arbitrary tonic. Rather than strip accidentals, it *equalizes* them — sharpening flat tonics or flattening sharp tonics on both arguments simultaneously, which is the Preservation Principle applied with α^{-1} — and then delegates to the white-key solver:

```
else if(tonic_accidental == "flat" && (tonic_letter %in% HAS_FLATS)){
    tonic <- tonic %ACC% "#"; note <- note %ACC% "#"   # transpose BOTH up
    return(.scaleDegreeWK(tonic, note))                # solve on white keys
}
```

Bridge Observation 8.2 (The theorem as the spec for a known bug). The residual branch `.complexCase()` (tonics like $C^\flat$ or $E^\sharp$, whose letter lacks the needed support) carries the confession *"doesn't quite do the trick; misses white keys..."* — it reaches for ad-hoc enharmonic cycling instead of same-accidental transposition, and the degree information decays in transit. The clean fix is to apply the Preservation Principle with the degree-aware $\tilde{d}$: reduce via same-accidental transposition, never via enharmonics, so the letter-degree skeleton survives. This is ledger item A6, and it is the purest example of the book repaying its debt to the code: the 2026 theorem is the specification for repairing the 2022 function.

9 Furry Cats, Fifths, and the Key-Signature Table

Book Theorem 9.1 (Furry Cat Theorem)**.** The largest chain of fifths requiring no accidentals is unique: the white-key sequence

$$\mathcal{F} : F - C - G - D - A - E - B \qquad (\text{"Furry Cats Get Dancey Around Every Bird"}),$$

with consecutive semitone distance $d(L_{i-1}, L_i) = 7 = \varphi$ throughout. The chain terminates at B because $d_5(B, F) = 6$: fixing it requires $F \mapsto F^\sharp$, which breaks the next fifth and forces the chain reaction that tiles the circle of fifths $\mathcal{F}^\flat - \mathcal{F}^\natural - \mathcal{F}^\sharp$.

The code never states the theorem — but it hard-codes its two consequences, and their *order of accrual*, in the key-signature table (`data.R`, `.KEYS_TABLE()`):

```
key_sigs <- c("nosf",
  "f#","f#c#","f#c#g#","f#c#g#d#","f#c#g#d#a#",
  "f#c#g#d#a#e#","f#c#g#d#a#e#b#",            # sharps: F C G D A E B
  "b-e-a-d-g-c-f-","b-e-a-d-g-c-","b-e-a-d-g-",
  "b-e-a-d-","b-e-a-","b-e-","b-")            # flats:  B E A D G C F
```

Sharps accrue in exactly the furry-cat order $\mathcal{F}$ (each new sharp is the next cat), and flats accrue in the reversed chain $B\,E\,A\,D\,G\,C\,F$ — the book's circle of fourths $\mathcal{R}$. A musician memorizes these strings; the theorem explains *why* these and no others are the fifteen key signatures. The book also derives the "cursed shell spellings" at the chain's seam — $\varphi(B^\flat) = F^\natural$, $\varphi(B^\natural) = F^\sharp$, $\varphi(B^\sharp) = F^{\sharp\sharp}$ — which is the single semitone gap $d_5(B, F) = 6$ wearing three different accidentals.

10 Enharmonics: Proper, Improper, and Simplification

Book Definition 10.1 (Chromatic Enharmonic)**.** The *proper* enharmonic pairs are the two spellings of each black key: $(A^\sharp, B^\flat), (C^\sharp, D^\flat), (D^\sharp, E^\flat), (F^\sharp, G^\flat), (G^\sharp, A^\flat)$. The white-key (*improper*) enharmonics are $(B, C^\flat), (E, F^\flat), (B^\sharp, C), (E^\sharp, F)$.

The code splits the definition into two operations along exactly the proper/improper seam:

```
# PROPER: cycle between a black key's two spellings
.cycle_enharmonic <- function(black_note){
  if(note_acc == "sharp"){ return((MUS_ALPH %STEPUP%   note_letter) %ACC% "b") }
  else if(note_acc == "flat"){ return((MUS_ALPH %STEPDOWN% note_letter) %ACC% "#") }
}
# IMPROPER: collapse a white-key spelling (Cb -> B, E# -> F, doubles too)
```

```
.simpler_enharmonic <- function(note){
  if(accidental == "flat" && !(note_letter %in% HAS_FLATS)){
    return( MUS_ALPH %STEPDOWN% note_letter )   # Cb -> B, Fb -> E
  }
  ...
}
```

The membership tests against `HAS_FLATS` / `HAS_SHARPS` are the book's condition $\text{Color}(L^\alpha) = \textbf{Wht}$: a flat spelling is improper precisely when the letter is not flat-supported. The shared word *improper* itself is part of the evidence — see §12. Where the book's definition is a static list of pairs, the code additionally makes simplification an *operation* (including double-accidental cases $F^{\flat\flat} \to E^{\flat}$), another instance of the implementation outrunning the formalization.

11 Tertiary Harmony: Where the Book Outruns the Code

Chapter 2's §*Tertiary Harmony* develops a quality algebra: qualities $\mathcal{Q} \in \{\Delta, \nabla\}$ classify skips by size ($d_3 = 3 \Rightarrow \nabla$, $d_3 = 4 \Rightarrow \Delta$); a *quality dyad* $\kappa_2\mathcal{Q}[L^\alpha]$ assigns an accidental pair to a root and its third; complementation satisfies the elegant

$$|\mathcal{Q}^c| = 7 - |\mathcal{Q}| \qquad (3 + 4 = 4 + 3 = 7),$$

so a triad is a dyad stacked with its complement, $\mathcal{Q}_3 = (L,\ \mathcal{Q}_2(L),\ \mathcal{Q}_2^c(T))$ — every white-key fifth splits as $\nabla + \Delta$ or $\Delta + \nabla$, and augmented chords "are not naturally occurring" between white keys.

Bridge Observation 11.1 (The one-way section). **No chord constructor exists anywhere in mistyR.** The core library can spell scales and classify degrees, but it cannot build a chord; the only chord-adjacent code, `one_chord_harms()` (`kern-module/R/kern-harmony.R`), *extracts* simultaneities from scores and diffs them mod 12:

```
one_chord_harms <- function(chord){
  ...
  for(i in 2:l){ intervals[i-1] <- (chord[i] - chord[i-1]) %% 12 }
  intervals
}
```

That is analysis, not construction. §*Tertiary Harmony* is therefore the one section of Chapter 2 that is *new mathematics* rather than retroactive formalization — theory written for code that does not exist yet. The missing implementation (a `dyad(L, quality)` / `triad(L, quality)` layer, built on `%ACC%` and the d_3 classifier, which the library already contains) is ledger item A9; landing it would make §2.3 retroactively true.

Fittingly, this is also where the strongest vocabulary fossil lives: both Definition *Quality Dyad* and Definition *Quality Triad* describe their accidental assignments as "an assignment of an **incidental** pair/triple" — the code's pre-2026 word surviving inside the book's most code-independent section.

12 Vocabulary Fossils: Evidence of Lineage

Three shared idiosyncrasies establish the direction of the arrow beyond reasonable doubt. None of them is standard music-theory or mathematical usage; each originated as a quirk of the 2022 code and surfaced in the 2026 manuscript.

1. **"Incidental."** The code's original universal term for accidentals: `incidental.R`, `INC_HASH`, `%INC%`, `.addIncidental()`. The live code was renamed to the standard *accidental* on 2026-07-10 (tag `pre-accidental-refactor` preserves the old vocabulary), but the book's Definitions *Quality Dyad* and *Quality Triad* still read "an assignment of an *incidental* pair (α, α_T)" — mistyR vocabulary fossilized inside the formalization.

2. **"Improper."** The code's `.improperStepDegree()` (`harmony.R`) handles tonic-letter-preserving alterations; the book classifies white-key enharmonics as *improper enharmonics* (Definition *Chromatic Enharmonic*). Same word, same seam of the same concept.

3. **The bold x.** The code encodes double-sharp as ASCII `"x"` because `##` confounded its regexes (the comment in `.ACC_HASH_EXT()` says so: *"removed `##` and `bb` due to regex confounding"*). The book writes the cursed-fifth spelling $\varphi(B^\sharp) = F^{\mathbf{x}}$ with a literal bold `x` — even though `preamble/commands.tex` defines a proper $\sharp\sharp$ macro (`\dsh`). An R regex workaround fossilized into the book's typography.[2]

13 From Theory to Stylometry: The Applied Layer

Everything above concerns the core library — Chapter 2's mathematics. The rest of the repository is what that mathematics is *for*: reading real scores and measuring their tonal style. This layer is outside Chapter 2's scope (prospective material for later chapters), but it consumes the Chapter 2 machinery at every step, so the bridge extends naturally.

13.1 The piece frame: one contract, two importers

Both importers produce the same tidy frame — piece columns `key_p`, `meter_p`, `measure_p`, plus one (`rhythmN`, `note_nameN`, `note_valueN`) triplet per simultaneous voice slot:

	`kern2df()` (`kern-module`)	`midi2df()` (`midi-module`)
Source	`.krn` text splines	`.mid` binaries (via `tuneR`)
Simultaneity	spline columns	equal attack ticks, bass in slot 1
Key	`*k[...]` signatures, windowed	Key Signature events, windowed
Spelling	as written in the score	key-aware per window (flat/sharp tables)
Since	2022 (from Palmer's `museR`)	2026 (parity importer, D2)

Because the contract is shared, every analysis function runs on either format unchanged — a mixed kern+MIDI corpus is now a frame-level triviality.

13.2 $\mathbb{Z}_{12}$ in the wild: note values

The frame's `note_value` column is the enharmonic collapse $\mathcal{L}^* \to \mathbb{Z}_{12}$ that Chapter 2 calls the semitone space — implemented as a 1-indexed scale rooted at $A^\flat$:

$$A^\flat = 1,\ A = 2,\ A^\sharp = B^\flat = 3,\ \ldots,\ G = 12,\ G^\sharp = 1 \text{ (wrap)}.$$

[2] Fixed on the book side on 2026-07-10 (commit `400c574`): the display now uses `\dsh`. The fossil is recorded here as history — it was live evidence at audit time.

Proper enharmonics share values by construction (the book's "twelve unique notes, so there are enharmonic spellings"). The kern importer reads these off a hash table (`KRN_NOTE_HASH`); the MIDI importer computes the same scale from MIDI pitch classes by conjugating $\mathbb{Z}_{12}$ through the root shift:

```
.m2df_note_value <- function(midi_note){ ((midi_note %% 12) + 4) %% 12 + 1 }
```

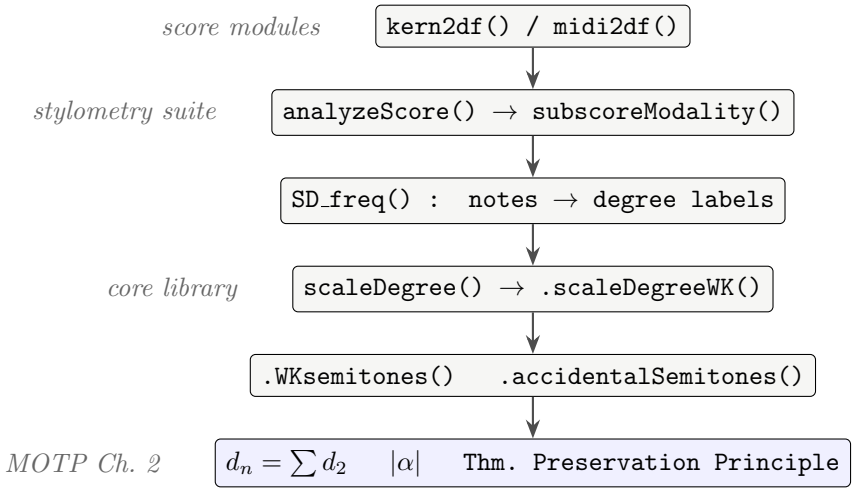
— the same shift-conjugation pattern as Bridge Observation 2.1, now aligning MIDI's C-rooted $\mathbb{Z}_{12}$ to misty's $A^{\flat}$-rooted $\{1, \ldots, 12\}$. And `one_chord_harms()` differencing consecutive chord tones mod 12 is interval arithmetic in exactly this space.

13.3 The stylometry chain runs on Chapter 2

The modality features — the repository's headline analysis — are computed per bar by classifying every note against the reigning tonic and counting membership in hand-chosen scale-degree sets:

Feature	Major keys counts	Minor keys counts
`blues`	$\{\flat 3, \sharp 4, \flat 5, \flat 6\}$	$\{3, \sharp 4, \flat 5\}$
`interchange`	$\{\flat 3, \sharp 4, \flat 6, \flat 7\}$	$\{\flat 2, 3, 6, 7\}$
`borrowed_sub`	$\{\flat 6, \sharp 5\}$	$\{6\}$
`consonance`	$\{1, 3, 5, 6\}$	$\{1, \flat 3, 5, \flat 6\}$
`dissonance`	$\{\flat 2, \sharp 2, \sharp 4, \flat 5, \flat 7, 7\}$ (shared)	

The degree labels those sets are matched against are *produced by* `scaleDegree()` — the Preservation-Principle reduction of §8 — whose accidental prefixes come from `.accidentalSemitones()`, the extended norm of §4. The dependency chain, top to bottom:



Every bluesiness score in `analysis/COMPOSERS.csv` is, at the bottom of its call stack, an application of the n-degree distance rule and the accidental norm. The applied layer is not adjacent to the theory — it is downstream of it.

Two further applied-layer constructs deserve book-side anchors when later chapters arrive: `relative_key()` (major-vs-relative-minor decided by a tonic-and-fifth frequency vote — an empirical procedure with no Chapter 2 analogue) and `DIATONIC_SCALES` (the mode list — Chapter 3's subject).

14 Convention Divergences

Five systematic differences of convention account for essentially all surface friction when reading one artifact with the other's vocabulary. None is a disagreement about content.

Axis	**mistyR**	**MOTP**
Letter indexing	1-indexed (`LETTERS[1:7]`; the `mod7`$(x-1)+1$ dance)	0-indexed $\mathbb{Z}_7$ ($A=0$)
Degree operator	`%STEP+% (degree - 1)`	$\Lambda_k = L \oplus (k-1)$
Flat symbol	kern `-` canonical; `b` accepted on input	♭ only
Double accidentals	first-class: `&`, `x`, norm ± 2	excluded from $\mathcal{A}$ (macros exist)
Step-distance criterion	flat-side only (`HAS_FL`)	dual flat/sharp formulation

One divergence is worth flagging as a live (mild) defect rather than a convention: the code is internally inconsistent about which flat symbol it *emits* — `enharmonic("F#")` returns `"Gb"` (letter `b`) while the spelling engine emits `"G-"` (kern), and only the kern form round-trips through the score-analysis vocabulary (`POSSIBLE_NOTES`). Unifying on kern `-` is ledger A12.

15 Asymmetries: What Each Side Has Alone

Book-only (formalism beyond the code)

- **Den keys and white neighbours** ($\overline{\textbf{Den}}$, $\overline{\textbf{WhtNeb}}$) — the intersection the code never takes (§5).
- **Rule of 9** — interval inversion pairing; the code never inverts (§3).
- **Furry Cat Theorem as a stated result** — the code hard-codes only its consequence, the key-signature table (§9).
- **The quality algebra** (Δ/∇ dyads, triads, complementation, stacking) — no constructor exists (§11).
- **The dual sharp-side step criterion** — added symmetry (§6).

Code-only (implementation beyond the book)

- **Double accidentals as first-class** — the norm extended to ±2 (§4; book remark pending, B14).
- **Enharmonic simplification as an operation** — `.simpler_enharmonic()`; the book only lists the pairs statically (§10).
- **The entire applied layer** — kern/MIDI parsing, the piece-frame contract, `relative_key()` voting, and the modality/rhythm/harmony stylometry (§13); prospective material for later chapters.

The pattern in these lists is the thesis of this paper in miniature: the book-only items are *structure* (symmetries, dualities, named abstractions, stated theorems) and the code-only items are *machinery* (edge cases, operations, I/O). That is precisely the signature of a formalization written after — and about — a prototype.

16 Maintenance and Companion Artifacts

This PDF is an expository *snapshot*, dated 2026-07-12. The durable, maintained mapping lives in two markdown companions inside the repository, and the division of labor is deliberate:

- `docs/motp-concordance.md` — the mapping itself: translation key, fossils, asymmetries, divergences. Anchored to function names and definition titles (never line numbers) so it survives edits on both sides. **Update it when either side renames anything**; it is the artifact, its prose is secondary.
- `docs/motp-bridge-todo.md` — the ledger: every insight that implies future work, with enough context to act on cold. Items cited in this paper: A6 (`.complexCase` via the Preservation Principle), A8 (den-key globals), A9 (dyad/triad constructors), A12 (flat-symbol unification), B7 (distance-family naming), B14 (double-accidental remark), D-track (module development).
- This paper (`docs/bridge-paper/`) — the explanation: derivations, worked examples, and the argument for the direction of the arrow. Recompile after major renames; regenerate rather than patch if the theory itself moves.

When MOTP's other chapters gain code counterparts (Ch. 1 ↔ tuning tables; Ch. 3 ↔ `DIATONIC_SCALES` and the mode stylometry), extend the concordance chapter by chapter (ledger C1) and add sections here rather than interleaving.

A The Complete Translation Key

The full symbol-to-construct mapping, in book order. (Maintained copy: `docs/motp-concordance.md` §1.)

Book notation	Meaning	Code construct
$\mathcal{L} = \{A, \ldots, G\}$	musical alphabet	`MUS_ALPH` (`data.R`)
$\mathbb{Z}_7$, $\oplus$ (looping)	letter arithmetic mod 7	`mod7()`, `.ALPH_RANGE()`, `.IDX_STATIC()`
$\Lambda_k(L) = L \oplus (k-1)$	k-degree letter	`%STEP+%`, `%STEPUP%`
$\mathbb{L}_k[L]$	k-degree letter sequence	`.ALPH_RANGE(note_index(L), k)`
$\mathcal{S}_7[L]$	heptatonic template	`.baseScale()` (`harmony.R`)
$\pi = (L, \alpha) = L^\alpha$	pitch label	note string (`"F#"`, `"B-"`)
Letter(π), Acc(π)	projections	`.dropAccidental()`, `.detectAccidental()`
$\pi \odot \alpha$	apply accidental	`%ACC%` / `.addAccidental()`
$(L^\sharp)^\flat = L$	neutralization	the `""` branch of `.addAccidental()`
$\lvert\alpha\rvert \in \{-1, 0, 1\}$	accidental norm	`.accidentalSemitones()` (± 2)
$\overline{\mathbf{Blk}}[\flat] = \{A, B, D, E, G\}$	flat-supported letters	`HAS_FL` / `HAS_FLATS`
$\overline{\mathbf{Blk}}[\sharp] = \{A, C, D, F, G\}$	sharp-supported letters	`HAS_SH` / `HAS_SHARPS`
Color(L^α)	key color	`.isBlackNote()` / `.isWhiteNote()`
$\overline{\mathbf{Den}} = \{A, D, G\}$, $\overline{\mathbf{WhtNeb}}$	den keys, white neighbours	(none — book-only; ledger A8)
proper / improper enharmonics	enharmonic pairs	`.cycle_enharmonic()` / `.simpler_enharmonic()`
d_2, d_3, d_5, d_n	white-key distances	`.WKsemitones(note, degree)`
Chromatic Algorithm	skeleton + borrowing	`.intervalNote()`
$q_k \in \mathbb{Z}_{12}$	degree $\to$ semitones	`SCALE_DEGREES`
Thm. Preservation Principle	transposition invariance	`scale()`'s structure; `scaleDegree()`'s reduction
$\mathcal{F}$ (furry-cat chain)	chain of fifths	`.KEYS_TABLE()` accrual order
$\mathbb{Z}_{12}$ collapse	enharmonic identification	`note_value` ($A^\flat = 1$); `KRN_NOTE_HASH`
Δ/∇ dyads, triads	quality algebra	(none — book-only; ledger A9)
Modes (Ch. 3)	diatonic modes	`DIATONIC_SCALES`

B Function Index

Where to find each function cited in this paper, with its book anchor.

Function	File	Book anchor (Ch. noted)
mod7, note_index, .ALPH_RANGE	R/data.R	Note $\mathcal{L} \cong \mathbb{Z}_7$
ALPH_HASH (HAS_FL/HAS_SH)	R/data.R	Defs. Black Flat-/Sharp-S
SCALE_DEGREES	R/data.R	interval table q_k
.KEYS_TABLE	R/data.R	Furry Cat Thm. (consequ
%STEPUP%, %STEP+%	R/scale.R	Def. k-Degree Letter
scale, .scaleBase	R/scale.R	Thm. Preservation Princi
diatonic_scale, blues_scale	R/scale.R	(applications)
%ACC%, .addAccidental	R/accidental.R	$\pi \odot \alpha$; Note Accidental N tion
.detectAccidental, .dropAccidental	R/accidental.R	Acc, Letter
.accidentalSemitones	R/accidental.R	Distance Rule Semitonal- Norm
.isBlackNote, .isWhiteNote	R/accidental.R	Color
enharmonic, .cycle_enharmonic, .simpler_enharmonic	R/accidental.R	Def. Chromatic Enharmo
.intervalNote	R/interval.R	Chromatic Algorithm
.WKsemitones	R/interval.R	Distance Rules Step / n-D
scaleDegree, .scaleDegreeWK	R/harmony.R	white-key sufficiency redu
.complexCase	R/harmony.R	(known bug; theorem is t A6)
.improperStepDegree	R/harmony.R	improper enharmonics (v
.baseScale	R/harmony.R	Def. Heptatonic Scale Ten
analyzeScore, subscoreModality, SD_freq, modality	R/analysis.R	(applied layer, §13)
kern2df, relative_key	kern-module/R/kern-import.R	(applied layer)
one_chord_harms, scale_degree_freq	kern-module/R/kern-harmony.R	$\mathbb{Z}_{12}$ interval arithmetic
rhythm_freq, rhythm_entropy, beat_density	kern-module/R/kern-rhythm.R	(applied layer)
midi2df and .m2df_* helpers	midi-module/R/midi-import.R	(applied layer; parity imp